

6 Implementing Logistic Regression using Python

```
# CH.SC.U4CSE24118 Ganath Avinash G R

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn import metrics
import seaborn as sn
```

Importing (Reading) Datasets

```
col_names = ['pregnant', 'glucose', 'bp', 'skin', 'insulin', 'bmi', 'pedigree', 'age', 'label']
df = pd.read_csv('../Data/diabet.csv', header=None, names=col_names)
print(df.shape)
df.head()
```

(768, 9)

	pregnant	glucose	bp	skin	insulin	bmi	pedigree	age	label
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Checking for any null values in dataset

```
print(df.columns)
df.isnull().sum()
```

```
Index(['pregnant', 'glucose', 'bp', 'skin', 'insulin', 'bmi', 'pedigree',
      'age', 'label'],
      dtype='str')
pregnant    0
glucose     0
bp          0
skin        0
insulin     0
bmi         0
pedigree    0
age         0
```

```
label      0
dtype: int64
```

✓ Assigning dependent and independent variables

```
feat_cols = ['pregnant', 'insulin', 'bmi', 'age', 'glucose', 'bp', 'pedigree']
x = df[feat_cols]
y = df.label
```

✓ Splitting the dataset into Training and Testing Dataset

```
x_tr, x_te, y_tr, y_te = train_test_split(x, y, test_size=0.2, random_state=5)
display(x_tr.shape, y_tr.shape, x_te.shape, y_te.shape)
```

```
(614, 7)
(614,)
(154, 7)
(154,)
```

✓ Fitting the Model (Logistic Regression)

```
mdl = LogisticRegression(solver='lbfgs', max_iter=1000)
```

```
mdl.fit(x_tr, y_tr)
y_pred = mdl.predict(x_te)
```

✓ Evaluation Metrics

```
conf_mat = metrics.confusion_matrix(y_te, y_pred)
print("Confusion Matrix: ", conf_mat)
acc_score = metrics.accuracy_score(y_te, y_pred)
print("Accuracy: ", acc_score)
print("Accuracy in percent: ", int(acc_score * 100), '%')
```

```
Confusion Matrix:  [[88 12]
 [19 35]]
Accuracy:  0.7987012987012987
Accuracy in percent:  79 %
```

✓ Confusion Matrix Heatmap

```
conf_mat = pd.crosstab(y_te, y_pred, rownames=['Actual'], colnames=['Predicted'])
sn.heatmap(conf_mat, annot=True)
```

<Axes: xlabel='Predicted', ylabel='Actual'>

